# Practical 7

## Part A:

## Aim:

**1. Form validator: define @NotBlank and @MaxLength(int) for fields and annotate a SignupForm. Write a checker that uses reflection to read each field's annotations and returns the list of validation errors for a given object.**

### Code:

NoteBlank.java

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface NotBlank {
}
```

Validator.java

```
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

public class Validator {

    public static List<String> validate(Object object) {

        List<String> errors = new ArrayList<>();

        Class<?> clazz = object.getClass();

        for (Field field : clazz.getDeclaredFields()) {

            field.setAccessible(true);

            try {
                Object value = field.get(object);

                // Check @NotBlank
                if (field.isAnnotationPresent(NotBlank.class))
{

                    if (value == null ||
                        value.toString().trim().isEmpty()) {

                        errors.add(field.getName() + " must not
be blank.");
                    }
                }

                // Check @MaxLength
                if (field.isAnnotationPresent(MaxLength.class))
{
```

```
                    MaxLength annotation =
                            field.getAnnotation(MaxLength.class
);

                    int maxLength = annotation.value();

                    if (value != null &&
                        value.toString().length() > maxLength)
{

                        errors.add(
                            field.getName()
                            + " must be at most "
                            + maxLength
                            + " characters."
                        );
                    }
                }

            } catch (IllegalAccessException e) {
                errors.add(
                    "Could not access field: " +
field.getName()
                );
            }
        }

        return errors;
    }
}
```

SignupForm.java

```
public class SignupForm {

    @NotBlank
    @MaxLength(20)
    private String username;

    @NotBlank
    @MaxLength(50)
    private String email;

    @MaxLength(15)
    private String mobile;

    public SignupForm(String username, String email, String
mobile) {
        this.username = username;
        this.email = email;
        this.mobile = mobile;
    }
}
```

MaxLength.java

```
public class SignupForm {

    @NotBlank
    @MaxLength(20)
    private String username;

    @NotBlank
    @MaxLength(50)
    private String email;

    @MaxLength(15)
    private String mobile;

    public SignupForm(String username, String email, String
mobile) {
        this.username = username;
        this.email = email;
        this.mobile = mobile;
    }
}
```

Driver.java

```
import java.util.List;

public class Driver {

    public static void main(String[] args) {

        SignupForm validForm =
                new SignupForm(
                    "riya",
                    "riya@gmail.com",
                    "9876543210"
                );

        SignupForm invalidForm =
                new SignupForm(
                    "",
                    "thisisanextremelylongemailaddress@example.
com",
                    "12345678901234567890"
                );

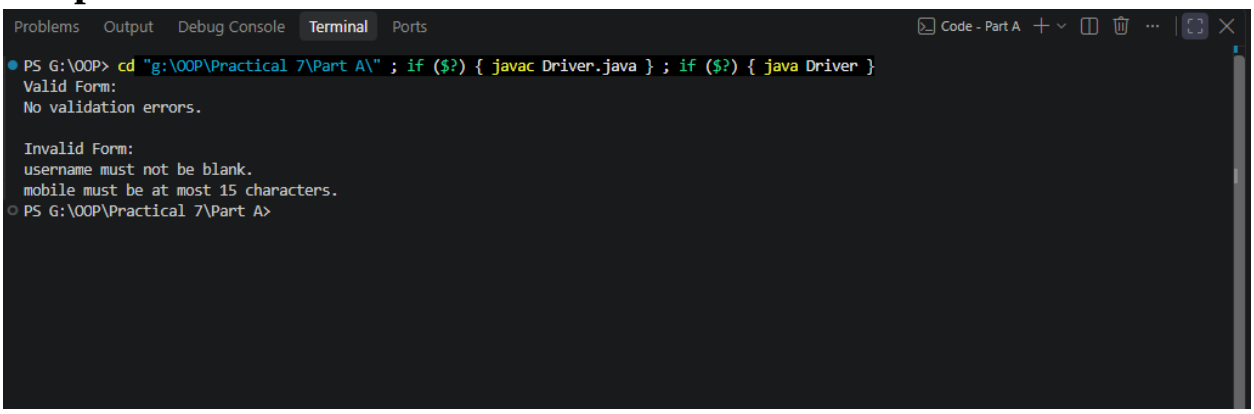
        System.out.println("Valid Form:");
        List<String> errors1 = Validator.validate(validForm);

        if (errors1.isEmpty()) {
            System.out.println("No validation errors.");
        } else {
            for (String error : errors1) {
                System.out.println(error);
            }
        }

        System.out.println("\nInvalid Form:");
        List<String> errors2 = Validator.validate(invalidForm);
```

```
        if (errors2.isEmpty()) {
            System.out.println("No validation errors.");
        } else {
            for (String error : errors2) {
                System.out.println(error);
            }
        }
    }
}
```

## Output:



**2. Mini test runner: define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods.**

## Code:

Run.java

```java
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Run {
}
```

SampleTests.java

```java
public class SampleTests {

    @Run
    public void testAddition() {
        System.out.println("testAddition passed");
    }

    @Run
    public void testSubtraction() {
        System.out.println("testSubtraction passed");
    }

    public void helperMethod() {
        System.out.println("This should NOT run.");
    }
```

```
    @Run
    public void testMessage() {
        System.out.println("testMessage passed");
    }

    public void normalMethod() {
        System.out.println("This should NOT run.");
    }
}
```

Driver.java

```
import java.lang.reflect.Method;

public class Driver1 {

    public static void main(String[] args) {

        SampleTests tests = new SampleTests();

        int count = 0;

        Method[] methods =
                tests.getClass().getDeclaredMethods();

        for (Method method : methods) {

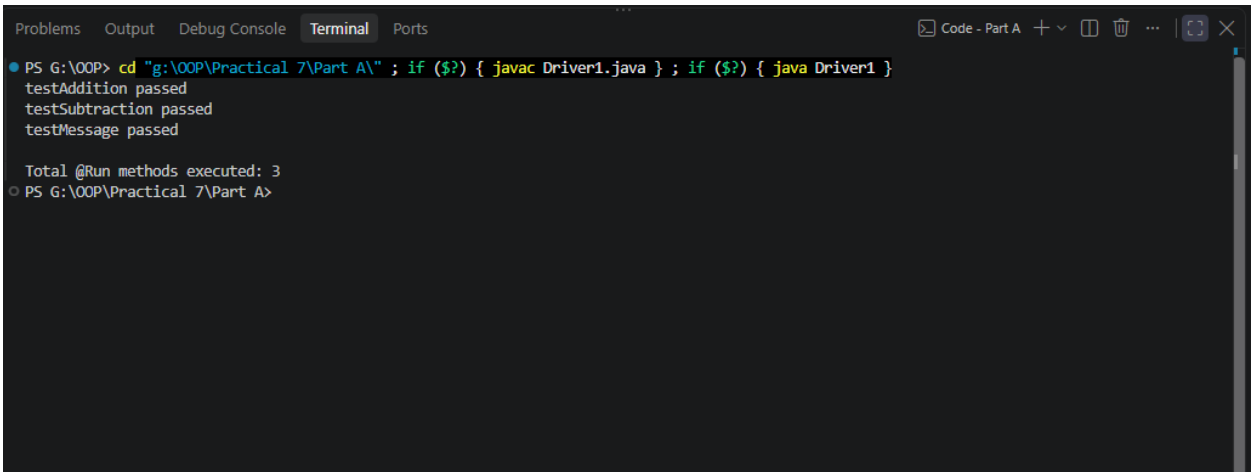
            if (method.isAnnotationPresent(Run.class)) {

                try {
                    method.invoke(tests);
                    count++;
                } catch (Exception e) {
                    System.out.println(
                        "Failed to run: " + method.getName()
                    );
```

```
                }
            }
        }

        System.out.println("\nTotal @Run methods executed: " +
count);
    }
}
```

## Output:



**3. (For Advanced Learners – home) Define a @Column(name) annotation and, by reflection, populate an object's fields from a header row and a data row by matching column names; decide how to handle a missing column.**

## Code:

Column.java

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Column {

    String name();
}
```

User.java

```
public class User {

    @Column(name = "name")
    private String name;

    @Column(name = "id")
    private String id;

    @Column(name = "email")
    private String email;

    @Override
    public String toString() {
        return "User{name='" + name
```

```
                + "', id='" + id
                + "', email='" + email + "'}";
    }
}
```

Mapper.java

```
import java.lang.reflect.Field;
import java.util.HashMap;
import java.util.Map;

public class Mapper {

    public static <T> T populate(
            Class<T> clazz,
            String[] headers,
            String[] data) throws Exception {

        if (headers.length != data.length) {
            throw new IllegalArgumentException(
                "Header and data lengths do not match."
            );
        }

        // Map column name -> value
        Map<String, String> values = new HashMap<>();

        for (int i = 0; i < headers.length; i++) {
            values.put(headers[i], data[i]);
        }

        T object =
clazz.getDeclaredConstructor().newInstance();

        for (Field field : clazz.getDeclaredFields()) {
```

```
            if (!field.isAnnotationPresent(Column.class)) {
                continue;
            }

            Column column =
                    field.getAnnotation(Column.class);

            String columnName = column.name();

            if (!values.containsKey(columnName)) {
                System.out.println(
                    "Warning: missing column '" + columnName +
"'"
                );
                continue;
            }

            field.setAccessible(true);

            String value = values.get(columnName);

            field.set(object, value);
        }

        return object;
    }
}
```

Driver.java

```
public class Driver2 {

    public static void main(String[] args) throws Exception {

        String[] headers = {
            "name",
            "id",
            "email"
        };

        String[] data = {
            "Riya",
            "A07",
            "riya@gmail.com"
        };

        User user =
                Mapper.populate(
                    User.class,
                    headers,
                    data
                );

        System.out.println(user);
    }
}
```

## Output:




## Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. What are the meta-annotations @Retention and @Target used for?

   **Ans.**

`@Retention` and `@Target` are **meta-annotations** because they are used to define how other annotations behave.

**`@Retention`**

It specifies **how long an annotation should be available**.

Example:
`@Retention(RetentionPolicy.RUNTIME)`

This means the annotation is available **at runtime**, so reflection can read it.

**`@Target`**

It specifies **where an annotation can be applied**.

For example:
`@Target(ElementType.FIELD)`

means the annotation can only be used on fields:

```
@NotBlank
private String username;
```

2. What is reflection, and what does getDeclaredFields() return?

**Ans.**

**Reflection** is a Java mechanism that allows a program to **inspect and interact with classes, fields, methods, constructors, and annotations at runtime**.

For example:
`Class<?> clazz = object.getClass();`

gets the runtime class of an object.

Then:
`Field[] fields = clazz.getDeclaredFields();`

returns an array containing the **fields declared directly inside that class**.

For example:

```
class SignupForm {

    private String username;
    private String email;
    private String mobile;
}
```

Calling:
`getDeclaredFields()`

returns `Field` objects representing:
`username`
`email`
`mobile`

It includes fields regardless of their access modifier, such as `private`, `public`, or `protected`.

3. How do annotations plus reflection let a framework validate any object?

**Ans.**

Annotations provide the **rules**, while reflection provides the **mechanism to discover and apply those rules**.

For example:

```
class SignupForm {

    @NotBlank
    @MaxLength(20)
    private String username;

    @NotBlank
    private String email;
}
```

The framework doesn't need to write separate validation code specifically for `SignupForm`.